

Digital Electronics / Computer Fundamentals: Understanding Base (Radix)

Computer Science ke starting semesters mein Number Systems sabse important foundation topic hota hai. Is guide mein hum bilkul basic se samajhenge ki Base kya hota hai, aur Base 10 (Decimal) ko Base 2 (Binary) mein kaise convert kiya jata hai.

1. Base (Radix) Kya Hota Hai?

Kisi bhi Number System mein **Base** (ya Radix) ka matlab hota hai ki us system mein total kitne unique symbols ya digits available hain numbers ko represent karne ke liye.

Kisi bhi system ka base hume do main baatein batata hai:

- Us system mein use hone wale total unique digits.
- Us system ke numbers ki positional values (weights) kis power mein badhengi.

Ab jab hume Base ka basic concept samajh aa gaya hai, chaliye dekhte hain ki hum apni daily life mein kaunsa Number System use karte hain.

2. Base 10: Decimal Number System

Yeh wahi number system hai jo hum bachpan se maths mein use karte aa rahe hain. Kyunki

humare paas counting ke liye ¹⁰ ungliyan hain, isliye shayad insaano ne is ¹⁰-base system ko sabse pehle adopt kiya.

- **Base:** ¹⁰
- **Digits Allowed:** Total ¹⁰ digits hote hain:

$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

- **Place Value (Positional Weight):** Har position right-to-left ¹⁰ ki power mein badhti hai ($10^0, 10^1, 10^2, \dots$).

Ek Real Example se Samajhte Hain:

Agar hum ek number likhte hain ³⁴⁵, toh iska actual matlab kya hota hai?

$$345_{10} = (3 \times 100) + (4 \times 10) + (5 \times 1) \\ = (3 \times 10^2) + (4 \times 10^1) + (5 \times 10^0)$$

Yahan har digit ki apni ek positional value hai jo 10 ke base par iska physical weight decide karti hai.

Hum humans toh Decimal Number System use karte hain, lekin computers ki physical working bilkul alag hoti hai. Isi wajah se computers ko ek alag Number System ki zarurat padti hai.

3. Base 2: Binary Number System

Computer ek electronic machine hai jo billions of transistors aur circuits se bani hai. Hardware level par, in circuits ke paas counting karne ke liye ungliyan nahi hoti. Unke paas sirf voltage levels hote hain. Isliye, circuits mein sirf do hi physical states ho sakti hain:

- **OFF** (No current / Low voltage) $\rightarrow$ Jise hum 0 se represent karte hain.
- **ON** (Current hai / High voltage) $\rightarrow$ Jise hum 1 se represent karte hain.

Isi physical limitation ki wajah se computers ke liye design kiya gaya **Binary Number System**.

- **Base:** 2
- **Digits Allowed:** Sirf 2 digits hote hain: $\{0, 1\}$ (Inhe hum **Bits** kehte hain - *Binary Digits*).
- **Place Value (Positional Weight):** Har position right-to-left 2 ki power mein badhti hai ($2^0, 2^1, 2^2, 2^3, \dots$).

Ab jab hume Decimal aur Binary dono Number Systems samajh aa gaye hain, agla logical step yeh dekhna hai ki ek system ke number ko doosre system mein kaise convert kiya jata hai.

4. Decimal (Base 10) ko Binary (Base 2) mein Kaise Convert Karein?

Decimal se Binary mein convert karne ke do simple aur popular methods hote hain. Dono ko step-by-step samajhte hain.

Method 1: Repeated Division by 2 (L.C.M. Method)

Yeh sabse standard aur foolproof method hai jo college exams mein marks score karne ke liye

use hota hai.

Rules:

1. Diye gaye Decimal number ko 2 se divide karein.
2. Jo Remainder (shesh) bache (jo ki ya toh 0 hoga ya 1), use side mein likh lein.
3. Jo Quotient (bhagfal) aaye, use fir se 2 se divide karein.
4. Yeh process tab tak repeat karein jab tak ki quotient 0 na ho jaye.
5. Final binary code nikalne ke liye saare remainders ko **Bottom-to-Top** (neeche se upar) ke sequence mein padhein aur likhein.
 - **MSB (Most Significant Bit):** Sabse neeche wala remainder, jiski value sabse zyada hoti hai.
 - **LSB (Least Significant Bit):** Sabse upar wala remainder, jiski value sabse kam hoti hai.

Step-by-Step Example 1: Convert 13_{10} into Binary ($?_2$)

Chaliye 13 ko divide karte hain:

Division Step	Quotient	Remainder	Position Type
$13 \div$	6	1	→ LSB (Upar wala)
$6 \div$	3	0	
$3 \div$	1	1	
$1 \div$	0	1	→ MSB (Neeche wala)

Neeche se upar (Bottom-to-Top) read karne par sequence milega: 1101

$$13_{10} = 1101_2$$

Step-by-Step Example 2: Convert 25_{10} into Binary ($?_2$)

Chaliye 25 ko divide karte hain:

Division Step	Quotient	Remainder	Position Type
$25 \div$	12	1	→ LSB
$12 \div$	6	0	
$6 \div$	3	0	
$3 \div$	1	1	
$1 \div$	0	1	→ MSB

Neeche se upar (Bottom-to-Top) read karne par sequence milega: 11001

$$25_{10} = 11001_2$$

Method 2: Powers of 2 (Weight/Addition Method)

Yeh fast mental calculation ka shortcut method hai jo objective exams ya coding rounds ke time bahut kaam aata hai.

- **Step 1:** Pehle right-to-left 2 ki powers ki values likh lein:

128 64 32 16 8 4 2 1

- **Step 2:** Jis number ka binary nikalna hai (jaise 13), dekhiye ki upar di gayi list mein se kaunse numbers ko add karke 13 banaya ja sakta hai. 13 banane ke liye hume chahiye:
 $8 + 4 + 1 = 13$
- **Step 3:** Jin numbers ka use hua hai unke neeche 1 likhein aur jinka use nahi hua unke neeche 0 likhein:

8	4	2	1
1	1	0	1

Aapka binary number ready hai: 1101_2

Is Method ko 25 par try karte hain:

Hume 25 banana hai. List mein se numbers select karein:

$$16 + 8 + 1 = 25$$

Ab binary bits assign karein:

16	8	4	2	1
1	1	0	0	1

Aapka binary number ready hai: 11001_2

Decimal se Binary conversion samajhne ke baad ab iska reverse process dekhte hain. Binary se Decimal conversion asal mein positional weights ke concept par based hota hai.

5. Binary (Base 2) ko Decimal (Base 10) mein Kaise Convert Karein?

Binary se Decimal convert karne ka logic bilkul simple hai: "**Weights and Addition**".

Jaise humne Base 10 mein dekha ki har position ka apna ek weight (10 ki power) hota hai, waise hi Binary (Base 2) mein har bit (0 ya 1) ka apna ek weight hota hai jo 2 ki power ($2^0, 2^1, 2^2, \dots$) par kaam karta hai.

Real-World Concept:

Maan lijiye aapke paas kuch digital switches hain aur har switch ek weight (value) ko control karta hai. Right side se shuru karte hain:

- Pehla Switch: Iski value hai 1 (kyunki $2^0 = 1$)
- Dusra Switch: Iski value hai 2 (kyunki $2^1 = 2$)
- Tisra Switch: Iski value hai 4 (kyunki $2^2 = 4$)
- Chautha Switch: Iski value hai 8 (kyunki $2^3 = 8$)
- ...aur yeh aage double hoti chali jayegi ($16, 32, 64, \dots$).

Rule bahut simple hai:

- Agar Switch 1 (ON) hai $\rightarrow$ Toh uski weight value ko final sum mein jodein (add).
- Agar Switch 0 (OFF) hai $\rightarrow$ Toh uski weight value ko ignore kar dein.

Step-by-Step Conversion Steps:

1. Sabse pehle diye gaye binary number ko thoda gap dekar likhein.
2. Har digit ke neeche/upar right-to-left 2 ki powers ki values likhna shuru karein ($1, 2, 4, 8, 16, \dots$).
3. Jis digit ke upar 1 hai, uski niche wali weight value ko select karein.
4. Jo bhi selected weight values hain, un sabhi ko aapas mein add kar dein. Aapka Decimal answer ready hai!

Step-by-Step Example 1: Convert 1101_2 to Decimal ($?_{10}$)

Chaliye humne jo pehle binary nikala tha (1101_2), use wapas cross-check karte hain.

- **Step 1: Bits aur unke Positional Weights ko assign karein (Right to Left):**
 - 1 (Right-most bit - LSB) $\rightarrow$ Weight: $2^0 = 1$
 - 0 $\rightarrow$ Weight: $2^1 = 2$
 - 1 $\rightarrow$ Weight: $2^2 = 4$
 - 1 (Left-most bit - MSB) $\rightarrow$ Weight: $2^3 = 8$

- **Step 2: Table ke zariye iski calculation ko dekhte hain:**

Binary Digit (Bit)	Weight (Base 2 Power)	Value	Multiply (Bit × Value)
1 _(MSB)	2 ³	8	1 × 8 =
1	2 ²	4	1 × 4 =
0	2 ¹	2	0 × 2 =
1 _(LSB)	2 ⁰	1	1 × 1 =

- **Step 3: Ab sabhi results ko add karein:**

$$\text{Decimal Value} = 8 + 4 + 0 + 1 = 13$$

Toh hum keh sakte hain:

$$1101_2 = 13_{10}$$

Step-by-Step Example 2: Convert 10110_2 to Decimal ($?_{10}$)

Chaliye thoda bada number lekar is concept ko aur solid karte hain.

- **Step 1:** Right-to-Left weights assign karein. Hume pata hai ki right se left weights double hote hain: 16, 8, 4, 2, 1 .
- **Step 2: Mapping aur Calculation:**

Binary Digit	Weight Value	Calculation
1	16 (2 ⁴)	1 × 16 =
0	8 (2 ³)	0 × 8 =
1	4 (2 ²)	1 × 4 =

1	$2 (2^1)$	$1 \times 2 =$
0	$1 (2^0)$	$0 \times 1 =$

- **Step 3: Final Addition:**

$$\text{Decimal Value} = 16 + 0 + 4 + 2 + 0 = 22$$

Toh hamara final binary to decimal conversion hoga:

$$10110_2 = 22_{10}$$

Ab tak humne dekha ki Binary hardware ke liye ideal hai. Lekin practical software development aur debugging ke perspective se Binary ke kuch practical drawbacks bhi hain. Inhi drawbacks ki wajah se higher-level representations ka use kiya jata hai.

6. Binary (Base 2) Ki Sabse Badi Dikkat: Hum Isse Aage Kyun Badh Rahe Hain?

Binary computer chips aur transistors ke physical level par kaam karne ke liye ekdum perfect system hai. Lekin jab baat aati hai software development, debugging, aur memory management ki, toh Binary mein kuch aisi badi dikkatein aati hain jiske chalte hume ek naye model ki zaroorat padti hai.

Dikkat 1: Visual Space aur Readability (Bohot Bade Numbers)

Binary system ka base bohot chhota (2) hai. Iska matlab hai ki sirf do hi digits (0 aur 1) hone ki wajah se, numbers ki length bohot tezi se badhti hai.

Ek simple decimal number 250 ko likhne ke liye decimal mein sirf 3 digits chahiye, lekin isko binary mein likhne ke liye hume 8 bits chahiye: 11111010_2 .

Modern computers mein memory addresses 32-bit ya 64-bit ke hote hain. Maan lijiye aapke RAM mein kisi memory block ka address decimal mein $4,294,967,295$ hai. Agar aapka debugging tool ise pure binary mein dikhayega, toh address aisa dikhega:

11111111111111111111111111111111₂

Itni lambi ³²-digit ki binary string ko screen par represent karna, visually padhna, aur track rakhna ek nightmare ban jata hai.

Dikkat 2: Human Error aur Eye-Strain (Galti Ki Sambhavna)

Insaani dimaag repetitive patterns (jaise lagatar ⁰ aur ¹) ko padhne mein bohot jaldi thak jata hai aur galti kar deta hai.

Maan lijiye aap ek firmware engineer hain jo hardware registers ko debug kar rahe hain, aur aapke samne do binary numbers aate hain:

Value A: 1111101111111111₂

Value B: 1111111111111111₂

In dono values ko ek jhalak mein dekh kar kya aap turant bata sakte hain ki dono mein kya farq hai aur kaunse position ka bit badla hai? Isme kafi dimaagi thakan lagti hai aur agar galti se ek bhi ¹ ki jagah ⁰ padh liya, toh system crash ho sakta hai.

Dikkat 3: Decimal (Base 10) Iska Solution Kyun Nahi Hai?

Aap soch rahe honge, "Toh binary ko chhodkar direct Decimal (Base 10) mein memory addresses kyun nahi likhte?"

Yahan par ek aur bada technical problem hai: Base 10 computer hardware ke structure se align nahi karta. Computer ki memory chunks powers of 2 par design hoti hain (jaise ⁸ bits = ¹ Byte, ¹⁰²⁴ Bytes = ¹ KB).

² ki kisi bhi integer power se hume kabhi bhi exact ¹⁰ nahi mil sakta (²³ = 8, ²⁴ = 16). Is wajah se Base 10 ko binary ke bits ke sath exact chunks mein correlate nahi kiya ja sakta. Agar hum memory ko Base 10 se map karenge, toh calculations bohot complex ho jayengi.

Hum Iska Kya Solution Nikalte Hain? (Concept of Grouping)

In sabhi dikkatkon ko door karne ke liye computer scientists ne ek smart approach lagayi:

"Kyun na hum bits ko chhote-chhote standard groups mein baant dein? Jaise,

4-bits ka ek group banayein ($2^4 =$)."

Agar hum ⁴ -bits ke har ek unique pattern ko ek single custom character ya symbol se represent kar dein, toh:

1. Humara ³² -digit ka lamba binary memory address simat kar sirf ⁸ **symbols** ka reh jayega!
2. Computer hardware ke units (⁸ -bit bytes) iske sath perfect mathematical ratio (^{1 : 2}) mein align honge.
3. Insaano ke liye screen par read karna aur debug karna behad aasan aur error-free ho jayega.

Yahi se ek aise naye base system ke janam ki shuruat hoti hai jo binary ki tarah hardware-friendly hai par decimal se bhi zyada compact aur visual hai.

7. Humne Base 16 (Hexadecimal) Hi Kyun Chuna? Base 10 Kyun Nahi?

Aapke dimaag mein ek bilkul natural sawaal aa sakta hai: **"Agar Base 10 sabhi insaano ko aata hai, aur Base 16 usse bhi bada aur naya hai, toh humne Base 10 ko chhodkar Base 16 (Hexadecimal) ko kyun adopt kiya?"**

Chaliye iske peeche ki behtareen mathematics aur computer architecture ko root level se samajhte hain.

1. Mathematical Misalignment (The 'Power of 2' Rule)

Computer ka hardware sirf **Base 2 (Binary)** par chalta hai. Is wajah se, hardware ke sath wahi system perfectly fit ho sakta hai jo ² **ki integer power** (2^n) mein aata ho.

Agar hum ² ki powers ko dekhein:

- $2^1 = 2$ (Binary)
- $2^2 = 4$ (Quaternary)
- $2^3 = 8$ (Octal)
- $2^4 = 16$ (Hexadecimal)
- $2^5 = 32$

Ab is list ko dhyān se dekhiye. Kya isme 10 aata hai? **Nahi!** 2 ki kisi bhi integer power (2^n) se hum 10 nahi bana sakte.

Kyunki 10 kisi power ka result nahi hai, isliye hum binary bits ko aise barabar groups mein nahi baant sakte jo perfectly Base 10 se map ho sakein.

2. Base 10 Ke Saath 'Grouping' Kyun Fail Ho Gayi? (Wastage of Space)

Maan lijiye humne zabardasti Base 10 use karne ka faisla kiya. Humne socha, "Hum binary bits ko group karke Decimal digits (0 se 9) represent karenge."

- **Agar hum 3-bits ka group banate hain:** Total combinations milenge $2^3 = 8$. Lekin hume Decimal ke liye 10 unique states (0 se 9) chahiye. Isliye 3 bits kam pad jayenge (8 aur 9 ko represent nahi kiya ja sakega).
- **Agar hum 4-bits ka group banate hain:** Total combinations milenge $2^4 = 16$. Hume chahiye sirf 10 states (0 se 9). Iska matlab hai ki baaki ki 6 states ($10, 11, 12, 13, 14, 15$) completely waste ho jayengi!

Agar hum physical level par har 4 bits ke chunk ko sirf $0 - 9$ represent karne ke liye limit kar dein (jise hardware mein *Binary Coded Decimal* ya BCD kaha jata hai), toh hardware ki efficiency bohot ghat jati hai aur calculation math bohot complex ho jati.

Iske opposite, **Base 16 (Hexadecimal)** pure $2^4 = 16$ states ko flawlessly use karta hai bina ek bhi state waste kiye!

3. Visual 1-to-1 Mapping (The Bite-Size Alignment)

Hexadecimal ka sabse bada jadoo yeh hai ki hum binary stream ko bina kisi complex calculation ke, sirf dhyān se dekh kar direct replace kar sakte hain.

Chaliye ek **8-bit register value (1 Byte)** ko dono systems mein compare karte hain:

Binary Value: 11001011_2

Case A: Hexadecimal (Base 16) ke sath representation:

Kyunki $16 = 2^4$, hum pure 8 bits ko perfectly do halves (4-bit chunks, jise **Nibbles** kehte hain)

mein divide kar sakte hain:



- Left Chunk (1100_2): Decimal value 12 hoti hai, jise Hex mein **C** likhte hain.
- Right Chunk (1011_2): Decimal value 11 hoti hai, jise Hex mein **B** likhte hain.

Toh bina kisi division ya multiplication ke, humne sirf dekh kar likh diya:

$$11001011_2 \rightarrow \mathbf{CB}_{16}$$

Yeh visual mapping itni direct hai ki hardware registers, memory addresses, aur coding levels par bits ko track karna aasan ho jata hai.

Case B: Decimal (Base 10) ke sath representation:

Agar hum isi 11001011_2 ko Decimal mein represent karna chahein, toh hum ise beech se divide nahi kar sakte kyunki 10 koi power of 2 nahi hai. Hume poora math calculate karna padega:

$$1 \times 128 + 1 \times 64 + 0 \times 32 + 0 \times 16 + 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 = \mathbf{203}_{10}$$

Sochiye, agar aapko lagatar 64-bit ka memory address decode karna ho, toh har baar poora calculation karna padega! Wahan decimal digit ko dekh kar aap binary bits ka andaza nahi laga sakte. Lekin Hexadecimal mein har ek character ($0 - F$) exact 4 binary bits ko visually reflect karta hai.

Feature / System	Decimal (Base 10)	Hexadecimal (Base 16)
Power of 2?	No ($2^n \neq$)	Yes ($2^4 =$)
Bit Grouping Alignment	Uneven chunks, hard to align	Perfect 4-bit boundary alignment
Wastage of Bits?	Yes, 6 states wasted in 4-bit group	No, all 16 states are used

Visual Decoding	Complex (Requires full multiplication)	Instant (1 Hex symbol = exact 4 bits)
-----------------	--	---------------------------------------

Isi zabardast coordination ki wajah se, Base 10 insaano ki ordinary life ke liye reh gaya, aur Base 16 computers ki high-level hardware/software communication ka standard ban gaya!

8. Hexadecimal System Aur Yeh A-F Ka Kya Chakkar Hai?

Ab aate hain us mudde par jisne aapko sabse zyada hairan kiya: **"Humne 0 se 9 ke baad achanak ginti chhodkar A-F alphabets kyun choose kiye? Aur in alphabets ki asali sachai kya hai?"**

Pehle samajhte hain ki Hexadecimal kya hai, aur fir iske symbols ke peeche ke dimaag-hila dene wale computer rules ko deconstruct karte hain.

1. Hexadecimal Number System Kya Hai?

- Hex = ⁶
- Deci = ¹⁰
- Hex + Decimal (Hexadecimal) = $6 + 10 = 16$

Iska seedha matlab hai ki is system ka **Base 16** hai, aur isme kisi bhi position par total **16 unique symbols** allowed hain.

2. single-digit Rule (Ek Position, Ek Hi Symbol!)

Maths ka ek strict rule hai: **Ek position (place value) par sirf ek hi akeli digit/symbol baith sakti hai.**

Hum Decimal mein likhte hain: ^{0, 1, 2, 3, 4, 5, 6, 7, 8, 9} . Yeh sab single symbols hain. Lekin ⁹ ke baad jab **Ten** (¹⁰) aata hai, toh hum do symbols use karte hain: a 1 and a 0.

Agar hum Hexadecimal mein ginti likh rahe hain aur hume ¹⁰ likhna hai, aur humne use direct 10 likh diya, toh ek bhayanak confusion paida ho jayega!

Confusion Dekhiye: Agar screen par likha hai:

Toh padhne wala confuse ho jayega ki:

- Kya yeh single position ka value **"Ten"** (10_{10}) hai?
- Ya fir yeh place value ke hisab se **"One-Zero"** ($1 \times 16^1 + 0 \times 16^0 = 16_{10}$) hai?

Is confusion ko khatam karne ke liye hume 10 se lekar 15 tak ki values ko represent karne ke liye naye, single-character symbols chahiye the.

3. Humne A-F Hi Kyun Chuna? Keyboard Ka Constraint!

Computer scientists ke paas do options the:

1. Ya toh woh bilkul naye symbols invent karte (jaise $\alpha, \beta, \star, \#$).
2. Ya fir pehle se maujood symbols ko reuse karte.

Sochiye, agar hum naye symbols invent karte, toh duniya ke har computer keyboard ko redesign karna padta! Har key par naye symbols chapne padte aur typing mushkil ho jati.

Isliye, sabse sasta aur smart rasta chuna gaya: Humare standard keyboards par pehle se maujood uppercase letters **A, B, C, D, E, F** ko use karna.

4. Alphabets Ki Value Ka Sach: Alphabetical Position vs. Hardcoded Values

Ab aapka sabse bada sawaal: "Kya 'A' ki value iske alphabet position se tay hoti hai (jaise Logical Reasoning mein $A=1, B=2, C=3$)? Ya computer iski apni alag value rakhta hai?"

Jawab hai: Logical reasoning ka rule yahan bilkul apply nahi hota!

Hexadecimal mein A ka matlab 1 ya 2 nahi hota. In alphabets ka asali kaam hai 9 ke baad ki counting ko carry forward karna. Kyunki 9 ke baad aane wali quantity **Ten** hoti hai, isliye A ki mathematical value directly 10 assign ki gayi hai.

Chaliye iski exact mapping ko dekh kar dimaag mein ek clear picture banate hain:

Hex Symbol	Decimal Quantity	Asali Matlab (Quantity)
9	9	Nau (Nine)
A	10	Das (Ten) — Not ¹ !
B	11	Gyaarah (Eleven)
C	12	Baarah (Twelve)
D	13	Teraah (Thirteen)
E	14	Chaudah (Fourteen)
F	15	Pandraah (Fifteen)

5. Computer Isse Kaise Samajhta Hai? (Inside the Silicon)

Physical level par, computer ke chips ko na toh 'A' pata hai aur na hi '10'. Computer chip sirf voltage states (**Binary: 0 aur 1**) samajhti hai.

Jab aap code likhte hain aur use compile karte hain, toh computer ka compiler/assembler aapke Hexadecimal characters ko parde ke peeche hardware registers ke liye **4-bit Binary pattern** mein convert kar deta hai:

- Jab aap writes hain **A** → Computer ke hardware mein automatically store hota hai: **1010** (which is $8 + 0 + 2 + 0 = 10$).
- Jab aap writes hain **F** → Computer ke hardware mein store hota hai: **1111** (which is $8 + 4 + 2 + 1 = 15$).

Conclusion: A se F tak ke letters sirf hum insaano ki aasan reading aur typing ke liye ek "mask" (chola) hain. Parde ke peeche computer unhe direct unke respective binary bits (¹⁰¹⁰ se ¹¹¹¹) ke roop mein store karta hai, jiska alphabetical positions se ^{0%} lena dena hai!

9. Hexadecimal Conversions: Har System Ke Beech Ka Bridge (Step-by-Step)

Ab tak humne conceptual baatein samajh li hain. Lekin exams mein aur practical programming mein hume in teen bases (**Base 2, Base 10, Base 16**) ke beech tezi se convert karna padta hai.

Chaliye sabhi conversion combinations ko bilkul root level se aur detail mein step-by-step clear karte hain.

PART A: Hexadecimal (Base 16) ko Binary (Base 2) mein Kaise Convert Karein?

Yeh conversion duniya ka sabse aasan aur instant conversion hai. Isme hume koi division ya multiplication nahi karna padta.

Super Rule:

- Diye gaye Hexadecimal number ke har ek character ko alag-alag split karein.
- Har ek single character ko uske **4-bit Binary (Nibble) equivalent** ke roop mein likh dein.
- **Caution:** Yaad rakhein, har symbol ko pure 4 bits mein hi represent karna hai. Jaise agar decimal value 2 hai, toh use binary mein sirf 10 nahi balki 0010 likhna padega.

Step-by-Step Example: Convert $3A5_{16}$ into Binary ($?_2$)

- **Step 1:** Diye gaye symbols ko door-door likhein:

3 A 5

- **Step 2:** Har symbol ke respective values aur 4-bit binary codes nikalte hain:

- Symbol **3**: Iska binary $2 + 1 = 3$ hota hai $\rightarrow$ 0011
- Symbol **A**: Iska matlab 10 hota hai, binary $8 + 2 = 10 \rightarrow$ 1010
- Symbol **5**: Iska binary $4 + 1 = 5$ hota hai $\rightarrow$ 0101

- **Step 3:** In teeno groups ko aapas mein line se jod kar ek sath likh dein:

0011 1010 0101

- **Step 4:** Left-most side ke extra zeroes ko hata kar final output ready karein:

$$3A5_{16} = 1110100101_2$$

PART B: Binary (Base 2) ko Hexadecimal (Base 16) mein Kaise Convert Karein?

Yeh Part A ka bilkul ulta (reverse) process hai. Yahan hum grouping method ka use karte hain.

Super Rule:

- Binary digits ko right side (**LSB - Least Significant Bit**) se shuru karke **4-4 bits ke groups** mein baantein.
- Agar sabse left wale group (MSB side) mein 4 bits poore nahi pad rahe hain, toh uske aage zero (0) laga kar group poora karein (**Padding**).
- Har 4-bit ke group ko uske single-digit Hex equivalent symbol mein convert kar dein.

Step-by-Step Example: Convert 110110111_2 **into Hexadecimal (** $?_{16}$ **)**

- **Step 1:** Sabse pehle right-to-left 4-bits ki grouping karte hain:

$$\dots 1 \quad \underbrace{1011}_{\text{Group 2}} \quad \underbrace{0111}_{\text{Group 1}}$$

- **Step 2:** Left side mein sirf single digit 1 bachi hai, use 4-bit banane ke liye aage teen zeroes pad karte hain:

$$\underbrace{0001}_{\text{Group 3}} \quad \underbrace{1011}_{\text{Group 2}} \quad \underbrace{0111}_{\text{Group 1}}$$

- **Step 3:** Ab har group ke physical weight ke hisab se hex symbol likhte hain:
 - Group 1 (0111): $4 + 2 + 1 = 7 \rightarrow$ Hex symbol: 7
 - Group 2 (1011): $8 + 2 + 1 = 11 \rightarrow$ Hex symbol: B (Kyunki Hex mein 11 ko B likhte hain)
 - Group 3 (0001): $1 \rightarrow$ Hex symbol: 1
- **Step 4:** Teen symbols ko ek sath jodein:

$$110110111_2 = 1B7_{16}$$

PART C: Hexadecimal (Base 16) ko Decimal (Base 10) mein Kaise Convert Karein?

Yahan hum use karte hain positional weight logic. Jaise humne Binary se Decimal banate waqt

2 ki powers ($2^0, 2^1, \dots$) ka multiplication kiya tha, theek waise hi yahan hum 16 ki powers (

$16^0, 16^1, 16^2, \dots$) ka multiplication karenge.

Super Rule:

- Right se Left shuru karte hue har position par $16^0, 16^1, 16^2$ upar likhein.
- Agar koi symbol letter (A-F) hai, toh pehle uski standard numeric value ($10 - 15$) likhein.
- Har symbol ko uske respective weight se multiply karein aur saare values ko aapas mein add kar dein.

Step-by-Step Example: Convert $2C5_{16}$ into Decimal ($?_{10}$)

- **Step 1:** Symbols ko separate karein aur unke weights likhein:
 - Digit 5 (Rightmost) $\rightarrow$ Weight: $16^0 = 1$
 - Symbol C (Middle) $\rightarrow$ Weight: $16^1 = 16$ (We know C's value = 12)
 - Digit 2 (Leftmost) $\rightarrow$ Weight: $16^2 = 256$
- **Step 2:** Weights aur values ko multiply karein:

$$\begin{aligned}\text{Decimal Value} &= (2 \times 16^2) + (C \times 16^1) + (5 \times 16^0) \\ &= (2 \times 256) + (12 \times 16) + (5 \times 1) \\ &= 512 + 192 + 5\end{aligned}$$

- **Step 3:** Sabhi values ko add karein:

$$\text{Decimal Value} = 512 + 192 + 5 = 709$$

- **Result:**

$$2C5_{16} = 709_{10}$$

PART D: Decimal (Base 10) ko Hexadecimal (Base 16) mein Kaise Convert Karein?

Yeh base-conversion ka standard repeated-division method hai. Yahan divisor hamesha hamara target base (yaani 16) hoga.

Super Rule:

- Diye gaye Decimal number ko 16 se divide karein.

- Remainder ko side mein record karein. Agar remainder ¹⁰ se ¹⁵ ke beech bache, toh use uske matching Hex character (A to F) mein convert kar dein.
- Jo Quotient bache use dobara ¹⁶ se divide karein.
- Yeh process tab tak karein jag quotient ⁰ na ho jaye. Saare remainders ko **Bottom-to-Top** order mein read karein.

Step-by-Step Example: Convert ⁴²⁹₁₀ into Hexadecimal ([?]₁₆)

- **Step 1: First Division**

$$429 \div 16 \rightarrow \text{Quotient} = 26, \text{ Remainder} = 13$$

Kyunki remainder ¹³ hai, iska Hex symbol hoga: D (LSB)

- **Step 2: Second Division** We divide quotient ²⁶ by ¹⁶:

$$26 \div 16 \rightarrow \text{Quotient} = 1, \text{ Remainder} = 10$$

Kyunki remainder ¹⁰ hai, iska Hex symbol hoga: A

- **Step 3: Third Division** We divide quotient ¹ by ¹⁶:

$$1 \div 16 \rightarrow \text{Quotient} = 0, \text{ Remainder} = 1$$

Remainder ¹ hai $\rightarrow$ Hex symbol: 1 (MSB)

- **Step 4: Align Remainders (Bottom-to-Top)**

Step	Division	Quotient	Remainder Value	Hex Symbol	Read Direction
1	$429 \div$	26	13	D	↑ LSB (Upar)
2	$26 \div$	1	10	A	↑
3	$1 \div$	0	1	1	↑ MSB (Neeche)

Neeche se upar padhne par hume milta hai: 1AD

- **Result:**

$$429_{10} = 1AD_{16}$$

10. Octal Number System (Base 8): Hum Hexadecimal Aur Decimal Ke Baad Base 8 Par Kyun Gaye?

Ab ek aisa sawaal jo aksar students ko pareshaan karta hai aur unki critical thinking ko trigger karta hai: **"Jab humare paas Decimal (Base 10) pehle se tha aur humne hardware efficiency ke liye Hexadecimal (Base 16) bhi bana liya, toh fir beech mein is Octal (Base 8) ko ghusane ki kya zarurat padi? Aisi kaunsi physical ya structural problem thi jo na toh Base 10, na Base 2, aur na hi Base 16 solve kar pa rahe the, aur use sirf Base 8 ne hi solve kiya?"**

Chaliye is mudde par critical thinking ke sath hardware level par jaakar asali sachai ko samajhte hain.

1. Historical Architecture Ka Sawaal: Word Size Aur Bits Ka Buntwara

Aapko yeh lag sakta hai ki computer hamesha se 8-bit bytes par chalte hain (jaise aajkal ke 32-bit ya 64-bit processors). Lekin computer history ke shuruaati daur (1960s aur 1970s) mein aisa bilkul nahi tha!

Uss daur ke mashhoor computers (jaise **PDP-8, PDP-11, ICL 1900, Honeywell mainframes**) alag hi architectures par chalte the. Unki word-sizes hote the:

12-bit, 18-bit, 24-bit, ya 36-bit

Ab chaliye critical analysis karte hain ki in machines ke liye systems kaise behave karte hain:

Case A: Agar hum in par Hexadecimal (Base 16) thopte:

Kyunki Hexadecimal ka base 16 (2^4) hai, isliye iska 1 digit hamesha **4 bits** ka group banta hai.

- Agar hum ek **18-bit machine** ke word ko Hex mein dikhana chahein:

$$\frac{18 \text{ bits}}{4 \text{ bits per Hex digit}} = 4.5 \text{ Hex digits}$$

4.5 digits! Iska matlab hai ki aap is 18-bit ke space ko clean Hex digits mein divide nahi kar sakte. Ek digit beech mein se half-cut ho jayegi! Yeh visually debug karne ke liye ek absolute disaster tha.

- Agar hum ek **12-bit machine** ko Hex mein represent karein, toh hume exact 2^3 digits milte hain ($2^3 \times 4 = 12$). Lekin, instruction formats mein 4-bit ka divisions set karna hardware logic ko bohot mehnga bana deta tha.

Case B: Aur tab champion bankar ubhra Octal (Base 8):

Octal ka base 8 (2^3) hai, jiska matlab hai ki iska 1 digit hamesha **exact 3 bits** ka group banta hai.

- **12-bit machine par:** $\frac{12 \text{ bits}}{3} = 4 \text{ Octal digits}$ (Perfect match!)
- **18-bit machine par:** $\frac{18 \text{ bits}}{3} = 6 \text{ Octal digits}$ (Perfect match!)
- **36-bit mainframe par:** $\frac{36 \text{ bits}}{3} = 12 \text{ Octal digits}$ (Perfect match!)

Bina kisi fraction ya data breakage ke, Octal system ne in systems ke data chunks ko perfectly coordinate aur display kiya.

2. Instruction Set (ISA) Alignment: The Register Mapping Rule

Programming mein compiler aur processor ke beech ki language ko samajhna bohot zaruri hota hai. Jab ek chip designer CPU design karta hai, toh woh instructions ko bits mein code karta hai.

Maan lijiye humare processor mein total **8 general-purpose registers** hain ($R0$ se $R7$ tak).

- In 8 registers ko select karne ke liye instructions ke andar hardware control unit ko kitne bits chahiye?

$$2^3 = 8 \rightarrow 3 \text{ bits}$$

Chaliye ek instruction ka format dekhte hain jo register-to-register copy karta hai: MOV [Destination Register], [Source Register]

Maan lijiye binary format aisa hai:



Ab is pure binary instruction (010101011) ko dono systems mein compare karte hain:

- **In Hexadecimal (Base 16):** 4-4 bits ke groups banayein: 10101011 $\rightarrow$ 01010101 ban jata hai 0AB or 1AB (padding ke baad). Kya aap is hex code AB ko dekh kar bata sakte hain ki

kaunsa register use ho raha hai? **Nahi!** Kyunki 4-bit ke group ne register selection ke 3-bit boundary ko aapas mein merge/cross kar diya hai. Register data instruction bits ke andar chhup gaya.

- **In Octal (Base 8):** 3-3 bits ke standard groups banayein:

$$\underbrace{010}_2 \quad \underbrace{101}_5 \quad \underbrace{011}_3 \rightarrow 253_8$$

Is Octal number 253 ko dhyan se dekhiye!

- Pehla digit 2 $\rightarrow$ Bata raha hai instruction type (MOV).
- Dusra digit 5 $\rightarrow$ Bata raha hai Destination Register ($R5$).
- Tisra digit 3 $\rightarrow$ Bata raha hai Source Register ($R3$).

Incredible! Octal ne binary instruction ko visually aisi transparency di ki machine code ko bina decode kiye screen par direct read kiya ja sakta tha.

3. Display Constraints: Nixie Tubes Aur 7-Segment displays Ki Purani Duniya

Aaj humare paas high-resolution IPS, OLED aur LCD screens hain jahan hum 'A' se 'F' tak ke alphabets bade aaram se behtareen style mein dekh sakte hain. Lekin 1960s ke lab hardware aur control panels ko yaad kijiye:

- **Nixie Tubes aur Neon Displays:** Uss daur mein displays gas-discharge tubes par chalte the jo sirf numbers (0 se 9) show kar sakti thi. Unmein alphabets ($A - F$) dikhana design wise impossible ya bohot zyada expensive hota hai.
- **7-Segment Displays:** Ek saste 7-segment display par 0 se 9 tak ke numbers clear dikhte hain. Lekin agar aapko Hexadecimal ka B dikhana ho, toh woh 8 jaisa dikhega. D dikhana ho toh 0 jaisa dikhega. Insaano ke liye is visual confusion ko handle karna ek bada headache tha.

Octal ne is problem ko poori tarah khatam kar diya! Kyunki Octal mein sirf 0 se 7 tak ke numbers use hote hain, isliye kisi bhi saste display unit par bina kisi confusion ya letter mapping ke data ko aaram se aur saste mein show kiya ja sakta tha.

4. Human Cognition: Letters Ka Dimaagi Bojh

Insaan ka dimaag numbers ke sath bohot comfortable hota hai. Hexadecimal padhte waqt jab aap C dekhte hain, toh aapke dimaag ko background mein ek extra task chalana padta hai: "C matlab 12".

Octal mein aisi koi alphanumeric jhanjhat nahi hoti. Yeh decimal ki tarah hi normal digits (0, 1, 2, 3, 4, 5, 6, 7) use karta hai, jisse insaano ke liye memory addresses aur codes ko bina kisi mathematical conversion ke directly padhna aur yaad rakhna kafi aasan ho jata hai.

Feature / System	Decimal (Base 10)	Hexadecimal (Base 16)	Octal (Base 8)
Bit-Grouping Size	N/A (Non 2^n system)	4 Bits per Digit	3 Bits per Digit
Best Suited For	Daily Human life, Math	8/16/32/64-bit modern computers	12/18/36-bit legacy/embedded systems
Visual Register Mapping	Poor	Average (breaks 3-bit registers boundary)	Excellent (flawless register representation)
Hardware Display Cost	Medium	High (requires letter support)	Very Low (runs on basic numeric displays)
Alphanumeric Overhead	No	Yes (requires $A - F$)	No (only standard digits $0 - 7$)

11. Octal Kya Hota Hai Aur Iske Conversions (Step-by-Step)

Ab hum dhyan se samajhte hain ki **Octal Number System (Base 8)** asal mein kaam kaise karta hai aur iske conversions ke piche kya logical flows hain.

1. Octal (Base 8) Ka Asali Matlab

"Octal" shabd Latin root octo se aaya hai, jiska matlab hota hai **Eight (8)**. Is system ka base ⁸ hai, yaani isme unique symbols ki sankhya strictly ⁸ hoti hai:

$\{0, 1, 2, 3, 4, 5, 6, 7\}$

SABSE BADA RULE: *Octal number system mein kabhi bhi digit '8' aur '9' exist nahi karte!* Jaise hum Base 10 (Decimal) mein ginti karte waqt ⁹ par pohochne ke baad single symbol khatam kar dete hain aur agla number ¹⁰ (One-Zero) likhte hain, theek waise hi Octal (Base 8) mein ⁷ ke baad single symbol khatam ho jate hain!

Toh Octal mein counting kaise hoti hai?

$0, 1, 2, 3, 4, 5, 6, 7, 10, 11, 12, 13, 14, 15, 16, 17, 20, \dots$

Quantity (Decimal)	Octal Symbol	Mathematical Proof (Place Value Weight)
7	7_8	$7 \times 8^0 =$
8	10_8	$1 \times 8^1 + 0 \times$ (Eight ka koi single digit nahi hota!)
9	11_8	$1 \times 8^1 + 1 \times$
16	20_8	$2 \times 8^1 + 0 \times 8^0 =$

Yeh concept samajhne ke baad conversions ka darr humesha ke liye khatam ho jayega. Chaliye ab ek-ek conversion ko crystal-clear mathematics ke sath solve karte hain.

PART A: Octal (Base 8) ko Binary (Base 2) mein Kaise Convert Karein?

Octal ($8 = 2^3$) ka ek digit hamesha **3 bits** ke barabar hota hai. Isliye iska rule bilkul Hex-to-Binary ki tarah simple hai:

Super Rule: *Diye gaye Octal number ke har ek single digit ko alag karein aur use strictly **3-bit Binary equivalent** ke roop mein likh dein.*

Step-by-Step Example: Convert ⁵²³₈ into Binary ([?] ₂)

- **Step 1:** Digits ko alag-alag likhein:

5 2 3

- **Step 2:** Har ek digit ko strictly 3-bit binary group mein badlein:

- Digit **5**: Iska binary $4 + 1 = 5 \rightarrow 101$

- Digit **2**: Iska binary $2 \rightarrow 010$ (Sirf 10 nahi likhna hai, 3 bits poore karne ke liye 010 likhenge!)

- Digit **3**: Iska binary $2 + 1 = 3 \rightarrow 011$

- **Step 3:** Teeno groups ko ek sath combine karein:

101 010 011

- **Result:**

$$523_8 = 101010011_2$$

PART B: Binary (Base 2) ko Octal (Base 8) mein Kaise Convert Karein?

Yeh Part A ka bilkul reverse process hai, jahan hum bits ki grouping karte hain.

Super Rule: Binary stream ko right side (**LSB**) se shuru karke **3-3 bits ke groups** mein baantein. Leftmost group mein agar 3 bits na ho toh aage zero (0) pad karein.

Step-by-Step Example: Convert 1011110_2 **into Octal (** $?_8$ **)**

- **Step 1:** Right to left 3-3 bits ke groups banayein:

...1 $\underbrace{011}_{\text{Group 2}}$ $\underbrace{110}_{\text{Group 1}}$

- **Step 2:** Left side ke single bit 1 ko 3 bits banane ke liye aage do zeroes add karein:

$\underbrace{001}_{\text{Group 3}}$ $\underbrace{011}_{\text{Group 2}}$ $\underbrace{110}_{\text{Group 1}}$

- **Step 3:** Har group ka respective numeric value likhein:

- Group 1 (110): $4 + 2 + 0 = 6$

- Group 2 (011): $0 + 2 + 1 = 3$

- Group 3 (001): $0 + 0 + 1 = 1$
- **Result:**

$$1011110_2 = 136_8$$

PART C: Octal (Base 8) ko Decimal (Base 10) mein Kaise Convert Karein?

Yahan hum ⁸ ki powers ke positional weights se multiplication karenge.

Super Rule: Right-to-left har position par $8^0, 8^1, 8^2$ ka weight assign karein, multiply karein aur aapas mein add kar dein.

Step-by-Step Example: Convert 245_8 into Decimal ($?_{10}$)

- **Step 1:** Right-to-left weights ko map karein:
 - Digit **5** (Rightmost) $\rightarrow$ Weight: $8^0 = 1$
 - Digit **4** (Middle) $\rightarrow$ Weight: $8^1 = 8$
 - Digit **2** (Leftmost) $\rightarrow$ Weight: $8^2 = 64$
- **Step 2:** Weights aur digits ka multiplication karke equation banayein:

$$\begin{aligned}\text{Decimal Value} &= (2 \times 8^2) + (4 \times 8^1) + (5 \times 8^0) \\ &= (2 \times 64) + (4 \times 8) + (5 \times 1) \\ &= 128 + 32 + 5\end{aligned}$$

- **Step 3:** Final addition karein:

$$\text{Decimal Value} = 128 + 32 + 5 = 165$$

- **Result:**

$$245_8 = 165_{10}$$

PART D: Decimal (Base 10) ko Octal (Base 8) mein Kaise Convert Karein?

Yahan hum repeated division method ka use karenge aur hamara divisor strictly ⁸ hoga.

Super Rule: Decimal number ko ⁸ se divide karte jao jab tak quotient ⁰ na ho jaye.
Remainders ko side mein likhein aur bottom-to-top padhein.

Step-by-Step Example: Convert 165_{10} **into Octal (**[?]₈**)**

- **Step 1: First Division**

$$165 \div 8 \rightarrow \text{Quotient} = 20, \text{ Remainder} = 5 \text{ (LSB)}$$

- **Step 2: Second Division**

$$20 \div 8 \rightarrow \text{Quotient} = 2, \text{ Remainder} = 4$$

- **Step-by-Step Example 3: Third Division**

$$2 \div 8 \rightarrow \text{Quotient} = 0, \text{ Remainder} = 2 \text{ (MSB)}$$

- **Step 4: Align Bottom-to-Top:**

Step	Division	Quotient	Remainder	Read Direction
1	$165 \div$	20	5	↑ LSB (Upar)
2	$20 \div$	2	4	↑
3	$2 \div$	0	2	↑ MSB (Neeche)

- **Result:** (Neeche se upar padhne par 245 milta hai, jo hamare Part C ke reverse verification ko perfectly match karta hai!)

$$165_{10} = 245_8$$

PART E: Octal (Base 8) ko Hexadecimal (Base 16) mein Kaise Convert Karein?

Yahan dhyan edin! Octal aur Hexadecimal ke beech **koi direct mathematically direct bridge**

nahi hota, kyunki 8 aur 16 ek dusre ki direct power nahi hain (jaise $8^2 \neq 16$).

Isliye hum use karte hain **Binary Bridge Method**, jo 100% foolproof hai aur bina complex division ke ho jata hai.

Octal (Base 8) $\xrightarrow{\text{3-bit Grouping}}$ Binary (Base 2) $\xrightarrow{\text{4-bit Grouping}}$ Hexadecimal (Base 16)

Step-by-Step Example: Convert 347_8 into Hexadecimal ($?_{16}$)

- **Step 1: Convert Octal to Binary (3-bits group)**

$$3 \rightarrow 011, \quad 4 \rightarrow 100, \quad 7 \rightarrow 111$$

$$\text{Binary Value} = 011100111_2$$

- **Step 2: Convert Binary to Hexadecimal (Regroup into 4-bits from right)**

Right to Left 4-bits ke pairs banayein:

$$\dots 0 \quad \underbrace{1110}_{\text{Group 2}} \quad \underbrace{0111}_{\text{Group 1}}$$

Left side par zero fill karein takki 4 bits poore hon:

$$\underbrace{0000}_{\text{Group 3}} \quad \underbrace{1110}_{\text{Group 2}} \quad \underbrace{0111}_{\text{Group 1}}$$

- **Step 3: Hex symbols mein values decode karein:**

- Group 1 (0111): $4 + 2 + 1 = 7$
- Group 2 (1110): $8 + 4 + 2 + 0 = 14 \rightarrow$ Hex symbol **E** (We know $14 = \text{E}$)
- Group 3 (0000): **0** (Hata sakte hain)

- **Result:**

$$347_8 = \text{E}7_{16}$$

PART F: Hexadecimal (Base 16) ko Octal (Base 8) mein Kaise Convert Karein?

Yeh Part E ka bilkul ulta process hai, aur isme bhi hum **Binary Bridge Method** ka hi use karenge.

Hexadecimal (Base 16) $\xrightarrow{\text{4-bit Grouping}}$ Binary (Base 2) $\xrightarrow{\text{3-bit Grouping}}$ Octal (Base 8)

Step-by-Step Example: Convert B_{16} into Octal ($?_8$)

- **Step 1: Convert Hex to Binary (4-bits group)**

We know $B = 11 \rightarrow$ binary $8 + 2 + 1 = 1011$

Digit $3 \rightarrow$ binary strictly 4-bits mein: 0011

Binary Value = 10110011_2

- **Step 2: Convert Binary to Octal (Regroup into 3-bits from right)**

Right to Left 3-bits ki grouping banayein:

... 10 $\underbrace{110}_{\text{Group 2}}$ $\underbrace{011}_{\text{Group 1}}$

Left side par 3-bits poora karne ke liye zero add karein:

$\underbrace{010}_{\text{Group 3}}$ $\underbrace{110}_{\text{Group 2}}$ $\underbrace{011}_{\text{Group 1}}$

- **Step 3: Octal values decode karein:**

○ Group 1 (011): $2 + 1 = 3$

○ Group 2 (110): $4 + 2 = 6$

○ Group 3 (010): 2

- **Result:**

$B_{16} = 263_8$

12. Summary Check

- **Decimal (Base 10):** Humari normal counting system (0 se 9 tak).
- **Binary (Base 2):** Computer ki language (0 aur 1).
- **Decimal to Binary Conversion:** 2 se tab tak divide karte jao jab tak quotient 0 na hache,

aur saare remainders ko neeche se upar ke order mein likh lo.

- **Binary to Decimal Conversion:** Har bit ko uske positional weight (power of 2) se multiply karke saare answers ko aapas mein jod (add) do.
- **Base 2 Limitation:** Binary numbers ki excessive length visual screen space ghere karti hai, errors badhati hai, aur standard memory alignments ke sath display karne ke liye directly efficient nahi hoti. Iska solution bits ki 4-bit grouping se nikalta hai.
- **Hexadecimal (Base 16) Transition:** 9 ke baad single position par double digit (10) nahi likh sakte, isliye keyboard ke standard letters A to F ko respectively 10 to 15 ki numeric value di gayi hai. Hardware level par, A hamesha raw binary pattern 1010 ko load karta hai.
- **Inter-Base Conversions:**
 - **Hex to Binary:** Har character ko 4-bit nibbles mein break karein.
 - **Binary to Hex:** Right side se 4-bit groups banayein aur target hex symbols mapping assign karein.
 - **Hex to Decimal:** 16 ki base-powers ($16^0, 16^1 \dots$) se multiply karein.
 - **Decimal to Hex:** 16 se repeated division karein aur remainders ko bottom-to-top track karein.
- **Octal (Base 8) Transition:** Legacy hardware (12-bit/18-bit), 3-bit CPU register structures aur purane saste numeric displays (jaise Nixie tubes) par bina $A - F$ letters ke visual binary mapping dene ke liye Base 8 ka avishkar kiya gaya tha.
- **Octal Conversions (Base 8):**
 - **Octal to Binary:** Har digit ko strictly 3-bit code (000 to 111) mein break karein.
 - **Binary to Octal:** Right side se 3-bits ke groups banayein.
 - **Octal to Decimal:** 8 ki powers ($8^0, 8^1 \dots$) ke positional weights se multiply karein.
 - **Decimal to Octal:** 8 se repeated division karein aur remainders bottom-to-top padhein.
 - **Hex to Octal & Octal to Hex:** Direct transition possible nahi hai, isliye **Binary Bridge** ka use karke flawless mapping nikalte hain.